

Project Design Report

Municipal Waste Billing System

CSE 3206 — Software Engineering Sessional | Lab 2
Group #06 — Section C (1st 30) | Selected Process Model: Waterfall

3. Project Title

Municipal Waste Billing System — a web application that digitizes resident registration and monthly waste-collection billing for a municipal ward office, built as a Flask REST API backend with a static HTML/CSS/JS frontend.

4. Problem Statement

Municipal solid-waste collection fees in many wards are still tracked using manual, paper-based registers. Ward clerks record resident details and calculate monthly waste-collection dues by hand, which is slow, error-prone, and makes it difficult to know which households have been billed, which bills are outstanding, and how much revenue has actually been collected in a given month.

Without a centralized system, the municipality has no reliable way to audit billing accuracy, follow up on unpaid dues, or generate collection summaries needed for budget planning. Residents also have no consistent record of their billing history. The client has therefore contracted our team to build a Municipal Waste Billing System that replaces the manual register with a simple, centralized web platform for registering residents and managing their waste-collection bills.

5. Project Objectives

- Provide a centralized digital platform for registering residents/holdings under a ward.
- Allow authorized municipal staff to generate and manage monthly waste-collection bills.
- Allow staff to record and track payment status (Paid / Unpaid) for every bill.
- Present an at-a-glance dashboard summarizing total dues, collections, and outstanding bills.
- Reduce manual paperwork and the calculation errors associated with a paper register.
- Build the system as a clean REST API + frontend so later labs can extend it with features such as notifications, an online payment gateway, and multi-tariff billing rules without reworking the core.

6. Stakeholder Analysis

The table below identifies the parties who are affected by, or who influence, the Municipal Waste Billing System.

Stakeholder	Role / Interest in the System
Municipality / City Corporation (Client)	Owns the system; needs accurate, timely billing and higher dues-collection rates.
Municipal Billing Staff (Clerks)	Primary end users; register residents, generate bills, and update payment status.

Stakeholder	Role / Interest in the System
Ward Supervisor / Administrator	Oversees billing operations for the ward and reviews collection summaries.
Residents / Rate Payers	Indirect stakeholders; receive bills and pay dues; benefit from accurate, transparent records.
System Administrator (IT support)	Maintains staff accounts and the hosting environment.
Development Team (2203136, 2203137, 2203138)	Analyzes requirements, designs, builds, tests and documents the system.

7. Functional Requirements

A minimum of ten functional requirements was required; thirteen were identified for this MVP:

1. The system shall allow staff to log in with a username and password.
2. The system shall allow a new staff account to be registered.
3. The system shall allow a logged-in staff member to log out, ending their session.
4. The system shall allow staff to register a new resident with name, holding number, address, phone number, and category (Household/Commercial).
5. The system shall allow staff to view a list of all registered residents.
6. The system shall allow staff to update an existing resident's details.
7. The system shall allow staff to delete a resident record.
8. The system shall allow staff to generate a new waste-collection bill for a registered resident, specifying billing month, amount, and due date.
9. The system shall allow staff to view a list of all generated bills.
10. The system shall allow staff to mark a bill as Paid or Unpaid.
11. The system shall allow staff to delete a bill record.
12. The system shall display a dashboard summarizing total residents, total bills, unpaid/paid bill counts, total amount due, and total amount collected.
13. The system shall expose all resident, billing, and dashboard functionality through a REST API restricted to authenticated staff sessions.

8. Non-functional Requirements

A minimum of eight non-functional requirements was required; nine were identified:

Category	Requirement
Usability	The interface shall be simple enough for municipal clerks with basic computer literacy to use with minimal training.

Category	Requirement
Performance	Pages and API responses shall load within 2 seconds under normal usage at MVP data scale.
Security	Passwords shall never be stored in plain text; they shall be stored using a one-way hash.
Reliability	Resident and bill records shall persist correctly across sessions using a durable database.
Maintainability	The backend REST API and the frontend shall be decoupled so either can be modified independently.
Portability	The backend shall run on any operating system with Python 3 installed; the frontend requires only a browser.
Availability	The system shall be accessible to authorized staff throughout municipal office hours with minimal downtime.
Scalability	The modular design shall allow the SQLite database to be replaced by a production RDBMS, and the static frontend to be deployed separately from the API, without redesigning the application.
Data Integrity	The system shall not allow a bill to be created for a resident that does not exist in the system.

9. Assumptions and Constraints

Assumptions

- Each resident/holding is billed at a flat, staff-determined rate per month; the MVP does not compute tariffs automatically.
- Billing staff have basic computer literacy and access to a web browser.
- The MVP will initially be piloted within a single municipal ward office.
- Actual payment collection (cash/mobile banking) happens outside the system; the MVP only records payment status.

Constraints

- The project is built by a 3-member team within a fixed semester lab schedule, so only MVP-level scope is implemented.
- SQLite, a lightweight file-based database, is used for the MVP and is not intended for concurrent multi-office production use.
- SMS/email notifications, an online payment gateway, and multi-tariff billing logic are out of scope for this milestone.
- The assigned process model for this project (Waterfall) requires requirements to be finalized upfront, with limited scope for change mid-milestone.

10. Selected Software Process Model

The Waterfall model was selected for the development of the Municipal Waste Billing System.

11. Justification of Process Model

- Requirements for a municipal billing system are well understood and standard (register residents, generate bills, track payment) and are unlikely to change significantly once agreed with the client.
- The client's domain (municipal administration) is process-driven and stable, which suits Waterfall's assumption of fixed, upfront requirements.
- With a small, fixed team of three members and a strict lab schedule, clear sequential phases (requirement analysis → design → implementation → verification → maintenance) are easier to plan and track than iterative ceremonies.
- Waterfall's phase-gate documentation style matches the lab's own structure (Phase 1 – Requirement Analysis, Phase 2 – Process Model Selection, Phase 3 – MVP Development, Phase 4 – GitHub Collaboration), so each lab milestone naturally maps to a Waterfall phase.
- The MVP scope (authentication API, backend REST API, frontend) was fully defined before development began, so extensive iteration was not required for this first milestone.

12. Comparison with Alternative Models

Two alternative models were considered and found less suitable for this project:

Agile Scrum

Scrum relies on sprint ceremonies — sprint planning, daily stand-ups, sprint reviews, and retrospectives — which add coordination overhead that is difficult to justify for a 3-member team delivering a single, already well-defined MVP milestone. Waterfall's linear structure maps more directly onto the lab's own Phase 1–4 breakdown, so Scrum's iterative cadence would add process overhead without a clear benefit at this stage.

Spiral

The Spiral model is intended for large, high-risk projects that need repeated rounds of risk analysis and prototyping. The waste billing MVP has low technical and business risk and a small, fixed scope, so Spiral's risk-driven iterative cycles would introduce unnecessary process overhead for a two-week lab milestone.

Rapid Application Development (RAD)

RAD depends on heavy, continuous user involvement and rapid prototyping cycles with a live client providing frequent feedback. In an academic lab setting without a client available for continuous review, RAD's core assumption does not hold, making it a poor fit compared to Waterfall's upfront requirement sign-off.

13. MVP Design Overview

The MVP follows a standard web-development split between a backend REST API and a frontend client, giving each team member a clearly separated, independently deployable part of the codebase — this maps directly onto the GitHub feature branches used in Phase 4.

Layer / Module	Responsibility	Owner	Feature Branch
Authentication API	Staff login, registration, logout, session-check endpoint, and the route guard reused by the backend API	2203136	feature/auth-module
Backend REST API	Resident CRUD, waste-bill CRUD, and the dashboard-summary endpoint, all returning JSON	2203138	feature/backend-module
Frontend	Static HTML/CSS/JS pages (login, register, dashboard, residents, bills) that call the REST API with fetch(); repository setup, README, and merging teammates' Pull Requests	2203137	feature/frontend-module

Architecture

The backend is a Flask application exposing a JSON REST API under `/api/*`, using session cookies for authentication. The frontend is a set of static HTML pages with vanilla JavaScript that calls those endpoints with `fetch()` and renders the results — no server-side templating is used, so the two layers only communicate through the documented API contract. For local development, Flask also serves the frontend's static files directly so the whole MVP can be run and demonstrated with a single command.

Technology Stack

- Backend: Python 3, Flask (application factory + Blueprints), REST-style JSON endpoints
- Database: SQLite via Flask-SQLAlchemy (ORM)
- Authentication: Session-based login with hashed passwords (Werkzeug security)
- Frontend: Static HTML, CSS, and vanilla JavaScript (fetch API) — no build tooling required for an MVP of this size

Data Model

The database has three entities:

- User — id, username, password_hash, role, created_at
- Resident — id, name, holding_number, address, phone, category, created_at
- Bill — id, resident_id (FK → Resident), billing_month, amount, status, due_date, created_at

A Resident has a one-to-many relationship with Bill: one resident can have many monthly bills, and every bill must reference an existing resident (enforced by a foreign key and validated in the API), which satisfies the data-integrity non-functional requirement.

Core Screens

- Login / Registration — staff authentication
- Dashboard — summary statistics and recent bills
- Residents — list, register, edit, delete residents
- Bills — list, generate, toggle paid/unpaid, delete bills

14. GitHub Collaboration Evidence

The team maintains a single shared GitHub repository (SWE_Lab_Project_2) following the branching workflow recommended in the lab handout:

- main — the integration branch; the shared foundation (app.py, extensions.py, models.py, requirements.txt) was committed here first.
- feature/auth-module (2203136) — authentication REST API.
- feature/backend-module (2203138) — resident and bill REST API.
- feature/frontend-module (2203137) — the static frontend.

Each member worked on their own feature branch, committed changes with meaningful, descriptive messages, pushed to GitHub, and opened a Pull Request against main. A teammate reviewed each Pull Request before it was merged; because the backend (auth/, api/) and the frontend (frontend/) live in separate folders, merge conflicts were rare and mainly limited to shared files such as backend/app.py, which were resolved by the branch owner before merging.

Repository Structure

- README.md — setup instructions, module ownership table, branching workflow, API summary
- docs/Requirement_Report.pdf — this Project Design Report, exported to PDF
- backend/auth/, backend/api/ — the authentication and backend REST API modules
- frontend/ — the static HTML/CSS/JS client
- assets/, screenshots/ — supporting media for GitHub documentation

Member Contribution Summary

Student ID	Primary Responsibility
2203136	Authentication REST API (login, registration, session handling, route guard); feature branch + Pull Request for auth-module
2203137	Frontend (all pages and JavaScript); repository creation, README, reviewing and merging teammates' Pull Requests into main, Project Design Report
2203138	Backend REST API for residents and bills (core MVP functionality); feature

Student ID	Primary Responsibility
	branch + Pull Request for backend-module

15. Challenges Encountered

- Agreeing on the database schema and the REST API contract (endpoint names, request/response JSON shape) before implementation began — a typical up-front planning challenge under the Waterfall model, resolved by finalizing the schema and endpoint list during the design phase before any module was coded.
- Keeping the frontend and backend properly decoupled — the frontend was built to depend only on the documented API responses, not on backend implementation details, so the two modules could be developed and tested in parallel.
- Ensuring every API route was consistently protected — solved with a single shared `login_required` decorator in the auth module that the backend API module imports and applies to every `resident/bill/dashboard` endpoint.
- Keeping the MVP scope small while still demonstrating authentication, a full REST API, and a working frontend within the lab's limited timeframe.

16. Conclusion

This lab took the Municipal Waste Billing System from a client problem statement through requirement analysis, process-model selection, and a working MVP. The requirement analysis identified thirteen functional and nine non-functional requirements and the relevant stakeholders. The Waterfall process model was selected and justified against Agile Scrum, Spiral, and RAD as better suited to a small, well-understood, fixed-scope milestone.

The resulting MVP — a Flask REST API backend split into authentication and core business-logic modules, paired with a static HTML/CSS/JS frontend — was implemented, tested end-to-end, and mapped onto a three-member GitHub workflow with one feature branch and Pull Request per member. This satisfies the lab's requirement that the repository history clearly demonstrate each member's contribution, and establishes a modular, API-driven foundation that future labs on design patterns and testing can build on directly.